

RUMMIKUB MEMORIA

Metodología y Programación
1PVG
Federico Sanjuan
Curso: 2025-2026
Andreu Sánchez Castelló



Índice/Index

1.- Database	03
Structure	03
Relations	03
2.- Tasks	04
Maria DB connection	04
ImGui	04
Use our graphic library	04
3.- Manual user	05
How to use our program	05
Main incorporations	06
4.- Post-Mortem	08



2. - TASKS

When we decide to make this project, we have clear then we want to use the MariaDB connections between or instance (Heidi MySQL on this case) and our client, which is our interface (graphic library), ImGui guide us to do it on this way.

Maria DB

This connection was made it by a connector with C / C++ with a 32 bits machine compiler, because our graphic library was compiled with a 32 bits machine too, thats why needs to be compatible.

Was hard to find a connector, because MariaDB right now is working on with a 64 bits machine, and old versions are not supported enough, but i found a repository on github for a guy who have a little maintenance of this.

If you wants to test this, you can have on our source code a file called [connection.cc](#), which if you compile and execute this you can check all steps for a mysql connection. The command to compile and run this executable is:

- compilacion c: [connection.cc](#)
- connection

Then you will have some logs on the console, which means the connection is open and closed correctly without errors.

ImGui

On other case we have ImGui, which is a interface helper for a graphical user interface, which have utilities to show some data on it.

We use this to show all tables, a good structure of all data inside them and some pagination too, this give us all what we need to compile every single step and every single table on them.

After all we use this to create some mixed queries inside our program to be sure all data is working all correctly.

Use our graphic library

Finally we have our ESAT library, which on this case is the middleware between the connection (MariaDB) and our client (ImGui). So thanks on this we can handle all data and show this one for our database, but being sure we dont call on each frame the same queries, if not this graphic library will collapse easily.



3. - MANUAL USER

For our knowledge, we are gonna show how you can use our program, being sure then all is tested and working on on a good platform for it.

In this case we use some technologies give it by the ESAT, like the graphic library and ImGui, but we just implement all the code needed to make like a middleware between them.

How to use our program

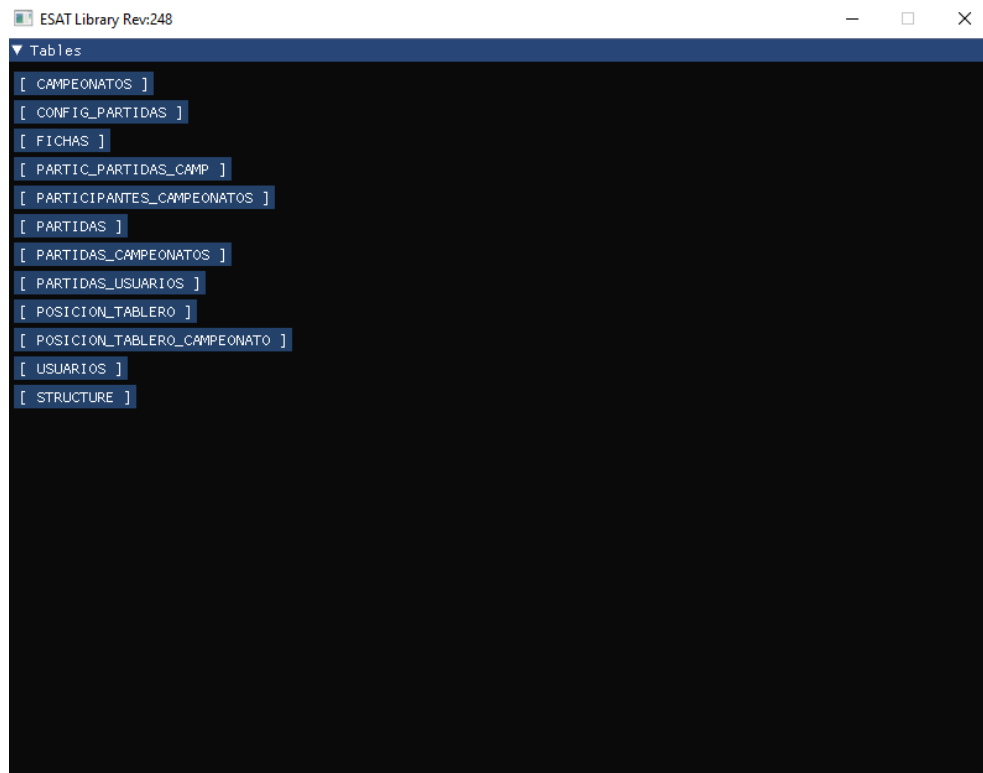
First of all, if we want to execute this, we need to try the connection at the beginning. Main issue is then we need to use the x86 visual studio console (developer prompt). If you put x86 on the windows search, this will appear, and then follow all the steps, going into the path of this project and do the commands below.

Credentials are setting on fire in code, but we have a file called [connection.cc](#) to just test all.

If we wants to go into main program, we need to compile and execute the [main.cc](#) file on the branch of this project:

- compilacion c: [main.cc](#)
- main

When we finish this, we should have something like this:



On this you can just see all tables data with a pagination and then if you click on one of them, you can insert, update or delete, it depends on what do you want to do, but for Insert, put a correct primary key, which you will need for the update one, and then delete for the same primary key value.



3. - MANUAL USER

The screenshot shows a database management interface. On the left, under the heading 'Tables', there is a list of tables: [CAMPEONATOS], [CONFIG_PARTIDAS], [FICHAS], [PARTIC_PARTIDAS_CAMP], [PARTICIPANTES_CAMPEONATOS], [PARTIDAS], [PARTIDAS_CAMPEONATOS], [PARTIDAS_USUARIOS], [POSICION_TABLERO], [POSICION_TABLERO_CAMPEONATO], [USUARIOS], and [STRUCTURE]. On the right, under the heading 'fichas', there is a table view for 'Table: fichas'. The table has three columns: FICHA_CODIGO, FICHA_COLOR, and FICHA_NUMERO. The data is as follows:

FICHA_CODIGO	FICHA_COLOR	FICHA_NUMERO
1	ROJO	1
2	ROJO	1
3	ROJO	2
4	ROJO	2
5	ROJO	3
6	ROJO	3
7	ROJO	4
8	ROJO	4
9	ROJO	5
10	ROJO	5

Below the table, there is a pagination bar showing '< Previous Page 1 / 11 Next >'. At the bottom, there are four buttons: INSERT, UPDATE, DELETE, and QUERY.

You can even make a query, but i never test using a query which include the modification of a configuration of x table, i recommend use the get info statement (SELECT FROM Some filters) or just insert, update or delete some registries.

Main Incorporations

We usually use printf to show all the debug errors on console, but on this case we prefer to use the STANDARD library of C++, std, which includes de std::cout and std::err. Reason is because this is more clear and for use, is better to include their the MySQL messages on the connection, or even SQL statements errors too.

Another things are methods like:

- strcpy_s, to copy a query including new info
- strcat_s, to concatenate a string with more info
- sprintf_s, to print on ImGui interface some info

Even using this, we dont need to specify which ImGui methods we use, but someones like, separator lines, or even begin a new window which some width and height include it, always being respectfully with the original margins for the main applications.

On special occasions, we need to reset memory because we change some screens, so methods like "memset" helps us a lot to clear memory, or even malloc to initialize it.

Maria DB connection gives us a lot of methods to prepare the query, execute and even fetch the results on possible data to show it, as you can see there:



3. - MANUAL USER

```
if (mysql_query(connection_mysql, query) != 0) {
    std::cerr << "[ERROR] Could not get table columns: " << mysql_error(connection_mysql) << std::endl;
    return 0;
}

MYSQL_RES* result = mysql_store_result(connection_mysql);

if (result == nullptr) {
    std::cerr << "[ERROR] Could not store columns result: " << mysql_error(connection_mysql) << std::endl;
    return 0;
}

MYSQL_ROW row;

int column_count = 0;

while ((row = mysql_fetch_row(result)) != nullptr) {
    if (column_count >= max_columns) {
        break;
    }

    columns[column_count].name = _strdup(row[0]);
    columns[column_count].type = _strdup(row[1]);

    column_count++;
}

mysql_free_result(result);

return column_count;
```

Method called `_strdup` is used to copy a memory dynamic of `char*`.

As i said, i try to be more confident on this code, so thats why i work using dynamic memory but collect it them as an array, which is what it is after all.

Even we use a class, called `Database`, which is the layer between connection and main code, here is where we include all SQL statements, including get tables data, columns and even mix of queries.



4. - POST MORTEM

Analize

After all i think was a pretty but small PHPMYAdmin what we have here, we can show dynamically all the data and even be sure this structure is working on.

Obviously we dont show anything as an static way, which i can said then structure screen was so difficult to visualize, thats why we only show the tables and their columns with types, but nothing more.

On last practices i dont even do anything particular with the global variables, i use to use them a lot, but i decide here just try to correct this, even know then all the booleans works as a flow control statements. I know this can be update as a major upgrade but was easier for me doing it on this way, even though all local functions can stablish all code there.

I need to mention the Database class (including their header, and their implementation). I prefer use this as a class because i want to separate everything on some forms, Database class, and main "class", Database include it for a layer on SQL statements and Main to take this data and modify their response,

After all i think we should use a cool way to implement everything, but i know some code of Main can be moved to another class or file, to just move logic unnecessary on main file too, but i decide not to move forward on it.